Transitive closure, and how to charge for it

the dl6 arc of 2026-08-06, as a paper

Every recursive dl6 rule is one shape. This is that shape, the two ways to charge for it, the recurrence a defect swapped in behind our backs, and the rail that grades a recurrence rather than an answer.

Workload throughout: `grid_10000`, 3,960 edges, 1,069,200 derived rows, checksum `9d7239568960d6a8`.

read this first

the spine, in one block

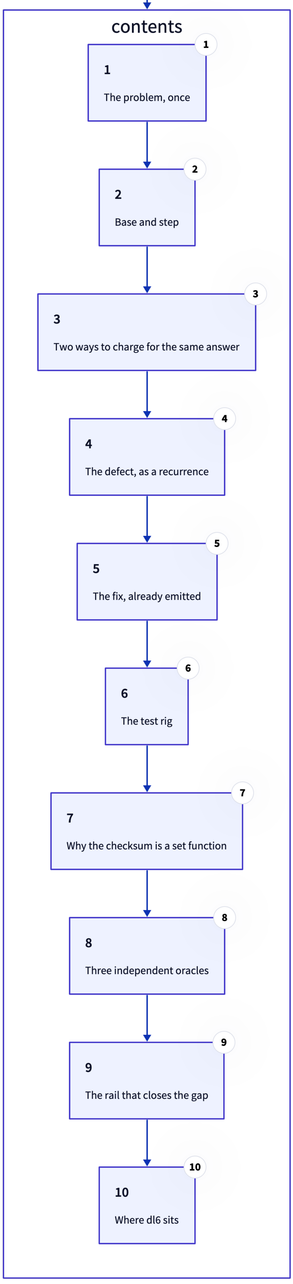
```
base       R0 = E
step       R(n+1) = Rn u (Rn o E)
fixpoint   R* = U Rn = Rk   for the least k with R(k+1) = Rk

naive      W = sum |Rn|.|E| = O(k.|R*|.|E|)
semi       W = sum |Dn|.|E| = O(|R*|.|E|)    since sum |Dn| = |R*|

defect     frontier_n = U(i<=n) Di = Rn  =>  W_actual = W_naive
rail       S(k) = O(1) one pass    vs    S(k) = O(k) round loop
```

The k in the cost sums and the k in the rail are the same k. That is why a statement count against depth catches what three answer-grading oracles all passed.

then walk it



Given

a finite directed graph

$$G = (V, E), \quad E \subseteq V \times V$$

Want

the reachability relation: every pair joined by a path of length one or more

$$R = E^{+} = \{(x, z) \mid \exists \text{ a path } x \rightsquigarrow z \text{ in } G\}$$

the two clauses ARE the base and the step

```
rel edge(source: int, target: int).
rel reachable(source: int, target: int).

reachable(Source, Target) <- edge(Source, Target).
reachable(Source, Target) <- reachable(Source, Mid),
                             edge(Mid, Target).
```

Why this one program

Every recursive dl6 rule is this shape with a different join. Fix the cost here and every recursive head inherits it.

Base

clause one seeds the relation with the edges themselves

$$R_0 = E$$

Step

clause two composes what is known with one more edge

$$R_{n+1} = R_n \cup (R_n \circ E) \quad \text{where} \quad R \circ E = \{(x, z) \mid (x, y) \in R,\ (y, z) \in E\}$$

$$R_0 \subseteq R_1 \subseteq R_2 \subseteq \cdots \subseteq V \times V$$

Termination

The chain is monotone and bounded above by a finite set, so it stops. It stops at the first repeat, and that repeat is the answer.

$$R^* = \bigcup_{n \geq 0} R_n = R_k \quad \text{for the least } k \text{ with } R_{k+1} = R_k$$

k is the graph's depth

family	k on the 10k case
`chain` (one long path)	2,582
`grid` (45 x 45 lattice)	88
`layered` (193 layers)	193

k is what a per-round evaluator pays for, and the families were chosen to spread it by three orders of magnitude.

Naive evaluation

Round n composes **everything known so far** with E.

$$W_{\text{naive}} = \sum_{n=0}^{k} |R_n| \cdot |E| \;=\; \Theta(k \cdot |R^*| \cdot |E|)$$

the fix is to shrink the left operand

Semi-naive evaluation

Round n composes only what round n-1 **newly** produced.

$$\Delta_n = R_n \setminus R_{n-1}, \qquad R_{n+1} = R_n \cup (\Delta_n \circ E)$$

$$W_{\text{semi}} = \sum_{n=0}^{k} |\Delta_n| \cdot |E| \;=\; \Theta(|R^*| \cdot |E|)$$

because $\sum_n |\Delta_n| = |R^*|$: each derived pair is new exactly once

The whole prize

The two differ by a factor of **k**, the recursion depth. On `grid_10000` , k = 88.

What the code claimed

`__frontier_reachable` holds the delta.

$$\texttt{frontier}_n \overset{?}{=} \Delta_n$$

```
SELECT DISTINCT d0."source", b0."target"
  FROM "__frontier_reachable" d0, "edge" b0
 WHERE d0."_phase" >= 0           -- admits every row
   AND b0."source" = d0."target" -- ever staged
```

What it actually held

Nothing retired the frontier between rounds, and the filter `_phase >= 0` admits every row ever staged.

$$\texttt{frontier}_n = \bigcup_{i \le n} \Delta_i = R_n$$

$$\Rightarrow\ W_{\text{actual}} = \sum_n |R_n| \cdot |E| = W_{\text{naive}}$$

Measured, 22 x 22 grid

pass n	new rows	frontier holds	ms
1	924	0	1.1
10	3,069	19,206	30.1
19	2,460	46,161	68.4
43	**0**	63,525	**87.6**

The compiler had written it months ago

`supportSql` carried the closure as one recursive query. `retractionGuardSql` only let it run when a delta carried a retraction, so a purely additive tick fell through to the round loop.

```
WITH RECURSIVE "reachable" ("source","target") AS (
  SELECT b0."source", b0."target"
    FROM "edge" b0                          -- base: R0 = E
  UNION                                     -- UNION, not UNION ALL:
  SELECT b0."source", b1."target"           -- dedup IS the delta
    FROM "reachable" b0, "edge" b1
   WHERE b1."source" = b0."target")         -- step: Rn o E
SELECT "source","target", 1 FROM "reachable";
```

Why `UNION` buys semi-naive for free

SQLite keeps the queue of rows it has yet to expand and drops duplicates on insert. A pair enters the queue exactly once, which is the same accounting as the delta.

$$\text{each } (x,z) \in R^* \text{ expanded once } \Rightarrow\ W = \Theta(|R^*| \cdot |E|) = W_{\text{semi}}$$

Measured, same input, same checksum

evaluation	ms	rounds
accumulating loop	100,809	89
frontier retired per round	4,452	89
one recursive CTE	**1,429**	**1**

Generator

`harness/src/gen.rs`, seeded and deterministic. Three families, parameters TUNED per scale so every case lands in the band.

$$10^6 \leq |R^*| \leq 2 \times 10^7$$

The three families isolate different k

family	shape	what it stresses
`chain`	one path, truncated	max depth, tiny fan-out
`grid`	2D lattice, right and down	mixed depth and fan-out
`layered`	DAG, adjacent layers only	join width, shallow

Every entrant speaks one wire contract

Three JSONL lines on stdout, nothing else, ever.


```
{"event":"loaded","edges":3960,"ms":17}
{"event":"fixpoint","derived":1069200,"ms":2776}
{"event":"done","checksum":"9d7239568960d6a8","peak_rss_kb":737584}
```

The lemma that makes paging sound

Comparing engines needs one number per ANSWER, and the answer is a **set**. Any digest that depends on enumeration order compares implementations rather than results.

$$c(R) = \bigoplus_{(x,y)\in R} h(x,y), \qquad h = \mathrm{fnv1a64}(x \| y)$$

$$\oplus \text{ is commutative and associative} \Rightarrow c \text{ is well defined on the set } R$$

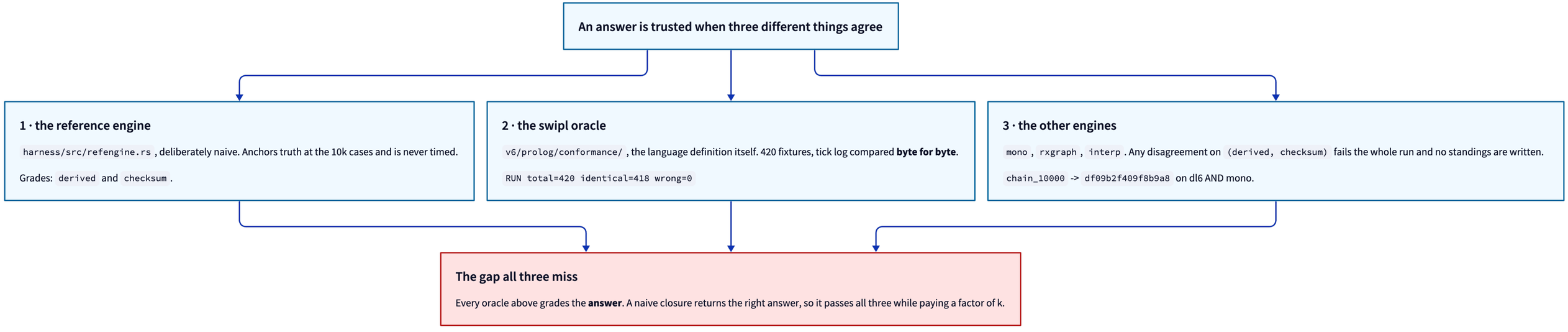
Corollary, used to fix a real OOM

Any partition of R folds to the same number, so the driver can read the head table in pages and drop each one.

$$R = P_1 \sqcup \cdots \sqcup P_m \Rightarrow c(R) = c(P_1) \oplus \cdots \oplus c(P_m)$$

holding 10M rows to XOR them
was the last OOM

```
// 250,000 rows per page, folded and dropped
SELECT "source","target" FROM "reachable"
 WHERE ("source","target") > (?, ?)
 ORDER BY "source","target" LIMIT 250000
```


An answer is trusted when three different things agree
1 · the reference engine
harness/src/refengine.rs , deliberately naive. Anchors truth at the 10k cases and is never timed.
Grades: derived and checksum .
2 · the swipl oracle
v6/prolog/conformance/ , the language definition itself. 420 fixtures, tick log compared byte for byte.
RUN total=420 identical=418 wrong=0
3 · the other engines
mono , rxgraph , interp . Any disagreement on (derived, checksum) fails the whole run and no standings are written.
chain_10000 -> df09b2f409f8b9a8 on dl6 AND mono.
The gap all three miss
Every oracle above grades the answer. A naive closure returns the right answer, so it passes all three while paying a factor of k.

Grade the RECURRENCE, not the answer

A one-pass closure issues a statement count independent of k. A per-round loop issues one insert plus its staging per round.

$$S(k) = \Theta(1) \text{ for one pass,} \qquad S(k) = \Theta(k) \text{ for a round loop}$$

```
// v6/tsv2/tests/recursiveClosureCounts.test.ts
const STATEMENTS_PER_TICK = 34;

assert.deepEqual(
  [shallow.statementCount,
   middle.statementCount,
   deep.statementCount],
  [34, 34, 34],
  "closure must not pay per round",
);
```

Fail-pre-fix receipt

Chain of n edges fed as ONE arrival batch, closure is n(n+1)/2 rows.

runtime	k=3	k=8	k=16
round loop	39	59	91
one pass	**34**	**34**	**34**

every engine computes the same R^* with the same $\Theta(|R^*| \cdot |E|)$ recurrence

So the ladder prices the CONSTANT, not the algorithm

`chain_10000`, 9,996,213 rows, `df09b2f409f8b9a8` on every row.

engine	rows/sec	constant is
`mono`	68,467,212	a u32 pair in a hash set
`rxgraph`	23,529,153	+ one virtual call per batch
`interp`	4,750,542	+ reading the rules per row
`dl6`	336,844	+ btree, WAL, planner, durability

What today actually removed

Zero of the 38.4x came from a better algorithm. All of it came from deleting work that bought nothing: a factor of k the frontier was charging, and rows that crossed into V8 only to be discarded.

What remains is the price of durability and generality, which is a thing to buy on purpose.